

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №7
по дисциплине «Методы оптимизации»
Вариант 11

Выполнил:
Бармичев Григорий Андреевич
Группа Р3210
Проверил:
Селина Елена Георгиевна

Постановка задачи

Кратко описать цель ЛР:

Цель работы — исследовать функцию двух переменных с седловой точкой, показать возможность «застревания» обычного градиентного спуска в окрестности седловой точки и сравнить это поведение с методом Adagrad. Также требуется применить библиотечный и собственный оптимизатор Adagrad для обучения нейронной сети.

Указать функцию варианта 11:

$$f(x, y) = 10^{-2}(8x^2 + 4xy + x + 4y - 7) \\ (x, y) \in [-20, 20] \times [-50, 50]$$

Задание 1. Аналитическое исследование функции

Задана функция:

$$f(x, y) = 10^{-2}(8x^2 + 4xy + x + 4y - 7), \\ (x, y) \in [-20, 20] \times [-50, 50].$$

Функция является многочленом двух переменных, поэтому она непрерывна и дифференцируема на всей области определения.

1. Стационарные точки

Найдём частные производные:

$$f'_x = 10^{-2}(16x + 4y + 1), \\ f'_y = 10^{-2}(4x + 4).$$

Стационарные точки определяются из системы:

$$\begin{cases} 16x + 4y + 1 = 0, \\ 4x + 4 = 0. \end{cases}$$

Из второго уравнения:

$$x = -1.$$

Подставим в первое:

$$16(-1) + 4y + 1 = 0, \\ 4y = 15, \\ y = \frac{15}{4} = 3.75.$$

Следовательно, единственная стационарная точка:

$$M_0 = (-1, 3.75).$$

2. Исследование стационарной точки

Найдём вторые производные:

$$f''_{xx} = 0.16, f''_{xy} = 0.04, f''_{yy} = 0.$$

Матрица Гессе:

$$H = \begin{pmatrix} 0.16 & 0.04 \\ 0.04 & 0 \end{pmatrix}.$$

Определитель:

$$\det H = 0.16 \cdot 0 - 0.04^2 = -0.0016.$$

Так как

$$\det H < 0,$$

матрица Гессе неопределённая. Значит, точка $(-1, 3.75)$ является седловой.

Других стационарных точек нет, поэтому локальных минимумов и максимумов внутри области нет.

3. Глобальный минимум

Рассмотрим функцию без положительного множителя 10^{-2} :

$$q(x, y) = 8x^2 + 4xy + x + 4y - 7.$$

Граница $x = -20$

$$q(-20, y) = 3173 - 76y.$$

Минимум при $y = 50$:

$$f(-20, 50) = -6.27.$$

Граница $x = 20$

$$q(20, y) = 3213 + 84y.$$

Минимум при $y = -50$:

$$f(20, -50) = -9.87.$$

Граница $y = -50$

$$q(x, -50) = 8x^2 - 199x - 207.$$

Вершина параболы:

$$x = \frac{199}{16} = 12.4375.$$

Значение функции:

$$f\left(\frac{199}{16}, -50\right) = -14.4453125.$$

Граница $y = 50$

$$q(x, 50) = 8x^2 + 201x + 193.$$

Вершина параболы:

$$x = -\frac{201}{16} = -12.5625.$$

Значение функции:

$$f\left(-\frac{201}{16}, 50\right) = -10.6953125.$$

Сравним значения: $-6.27, -9.87, -14.4453125, -10.6953125$.

Минимальное значение: -14.4453125 .

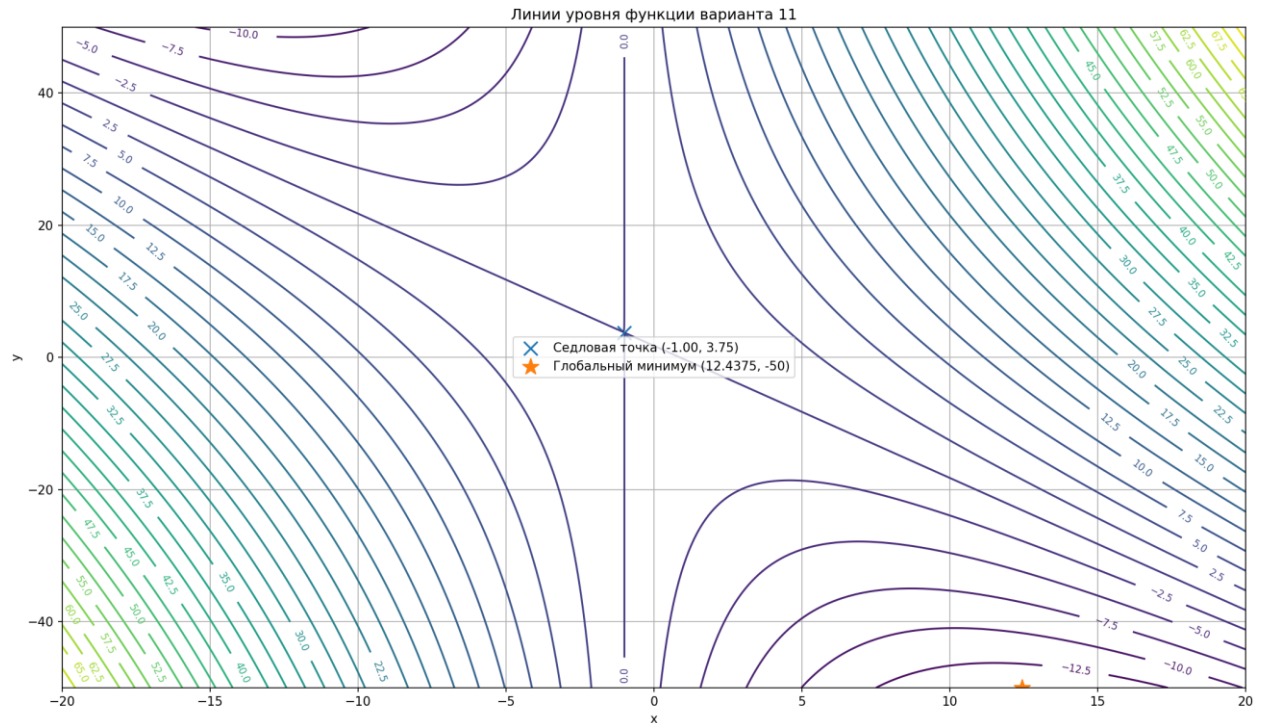
Следовательно, глобальный минимум достигается в точке: $\left(\frac{199}{16}, -50\right) = (12.4375, -50)$

$$f_{\min} = -14.4453125$$

4. Линии уровня

Для функции были построены линии уровня на области $[-20, 20] \times [-50, 50]$.

На графике отмечены седловая точка $(-1, 3.75)$ и глобальный минимум $(12.4375, -50)$.



Линии уровня показывают, что в окрестности стационарной точки функция в одних направлениях возрастает, а в других убывает, что соответствует седловой точке.

Вывод по заданию

Функция имеет единственную стационарную точку: $(-1, 3.75)$. Она является седловой, так как $\det H = -0.0016 < 0$. Локальных экстремумов внутри области нет. Глобальный минимум достигается на границе области: $(12.4375, -50)$, $f_{\min} = -14.4453125$. Наличие седловой точки создаёт сложность для оптимизационных алгоритмов: в её окрестности градиент близок к нулю, поэтому обычный градиентный спуск может замедляться или «застревать».

Задание 2. Исследование методов оптимизации

Дана функция:

$$f(x, y) = 10^{-2}(8x^2 + 4xy + x + 4y - 7).$$

Из задания 1 известно, что функция имеет седловую точку: $M_s = (-1, 3.75)$, а глобальный минимум достигается на границе области: $M_{\min} = (12.4375, -50)$.

2.1. Обычный градиентный спуск

Для обычного градиентного спуска использовалась формула:

$$z_{k+1} = z_k - \alpha \nabla f(z_k), \text{ где } z_k = (x_k, y_k).$$

Градиент функции: $\nabla f(x, y) = 10^{-2} \begin{pmatrix} 16x + 4y + 1 \\ 4x + 4 \end{pmatrix}$.

Начальное приближение было выбрано так, чтобы траектория сходилась к седловой точке:

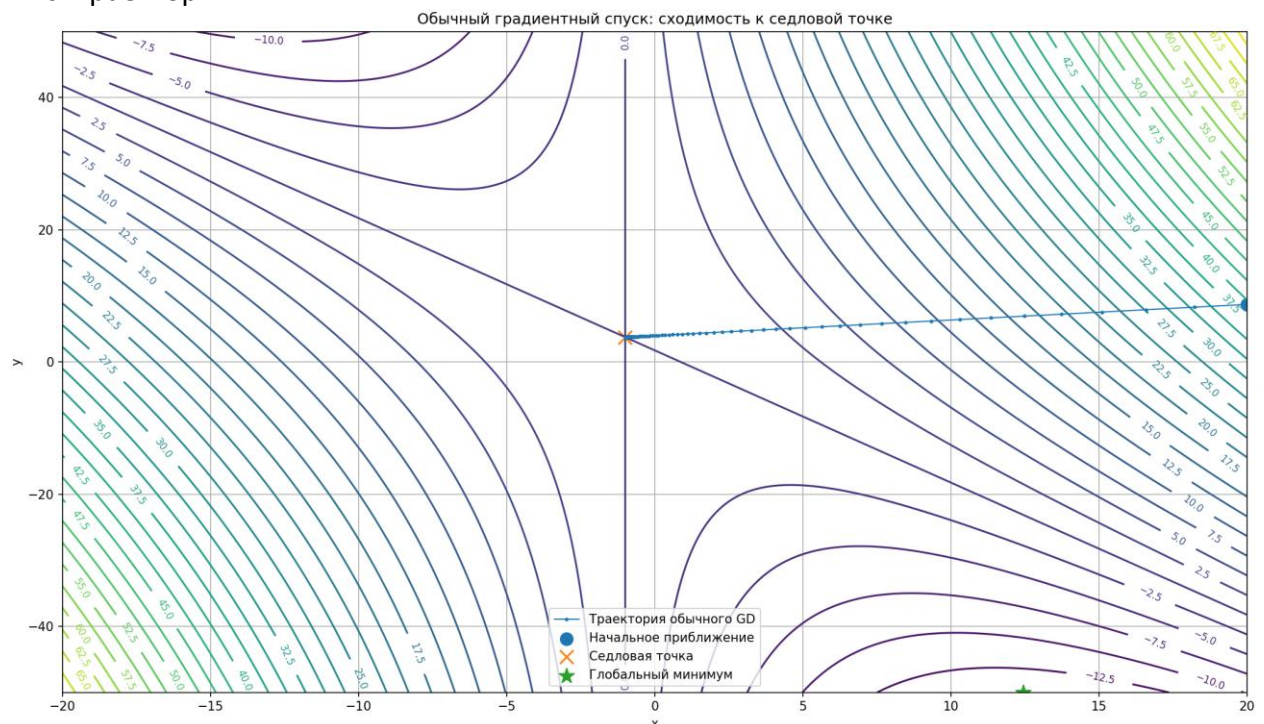
$$(x_0, y_0) = \left(20, \frac{15}{4} + 21(\sqrt{5} - 2)\right) \approx (20, 8.7074).$$

Параметры градиентного спуска:

$$\alpha = 0.5, \quad \varepsilon_{\nabla f} = 5.5 \cdot 10^{-4}, \quad \varepsilon_f = 1.7 \cdot 10^{-7}.$$

Критерии останова: $\|\nabla f(x_k, y_k)\| < \varepsilon_{\nabla f}$

Рис. Траектория.



Последовательные приближения сошлись в окрестность седловой точки: $(-1, 3.75)$.

Обычный градиентный спуск сошёлся к седловой точке, а не к глобальному минимуму. Это показывает проблему «застревания» градиентного спуска в окрестности седловых точек.

2.2. Собственная реализация Adagrad

Далее был реализован метод Adagrad. В отличие от обычного градиентного спуска, Adagrad использует адаптивную по координатам скорость обучения.

Формулы метода:

$$G_{k+1} = G_k + g_k^2,$$

$$z_{k+1} = z_k - \frac{\alpha}{\sqrt{G_{k+1}} + \varepsilon} g_k.$$

Квадрат градиента и деление выполняются по координатам.

Использовались те же начальная точка и количество итераций:

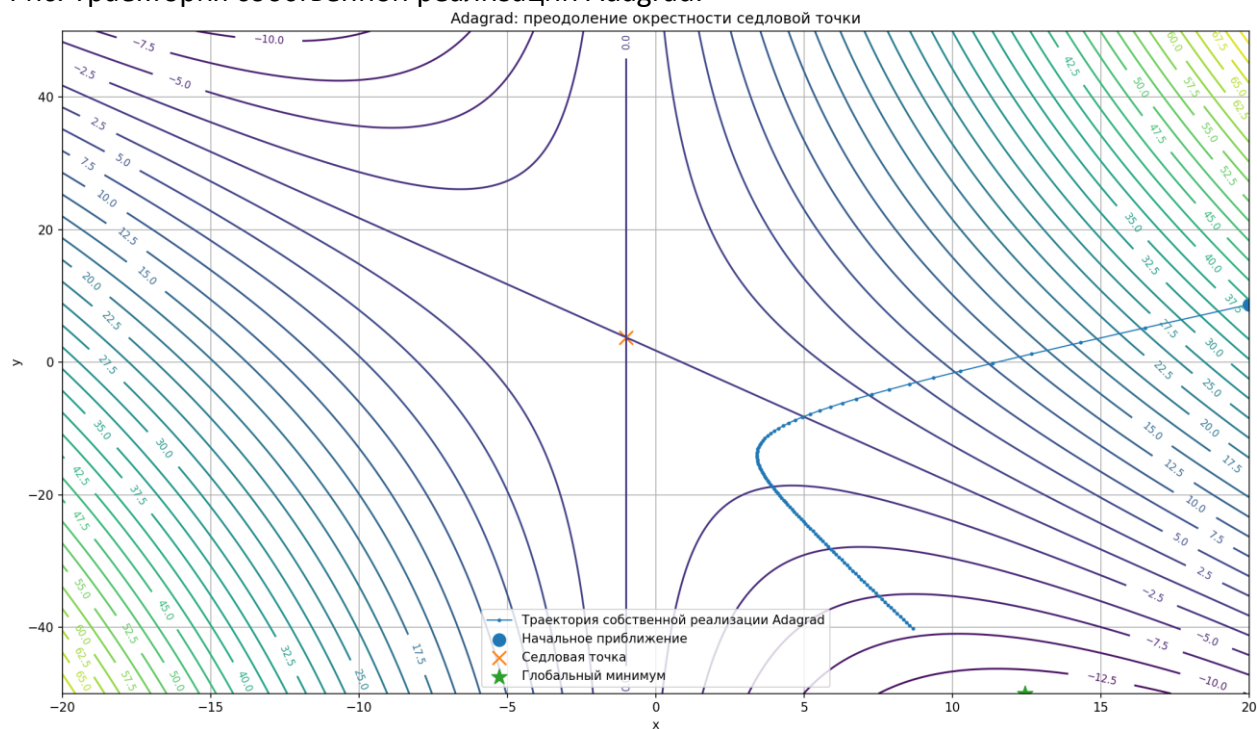
$$(x_0, y_0) \approx (20, 8.7074), N = 100.$$

Параметры Adagrad:

$$\alpha = 3.5, \quad \varepsilon = 10^{-8}.$$

В результате собственная реализация Adagrad не остановилась в окрестности седловой точки, а продолжила движение в сторону области глобального минимума.

Рис. Траектория собственной реализации Adagrad.



Метод Adagrad смог преодолеть окрестность седловой точки. Это связано с тем, что алгоритм изменяет скорость обучения отдельно для каждой координаты с учётом накопленной истории квадратов градиентов.

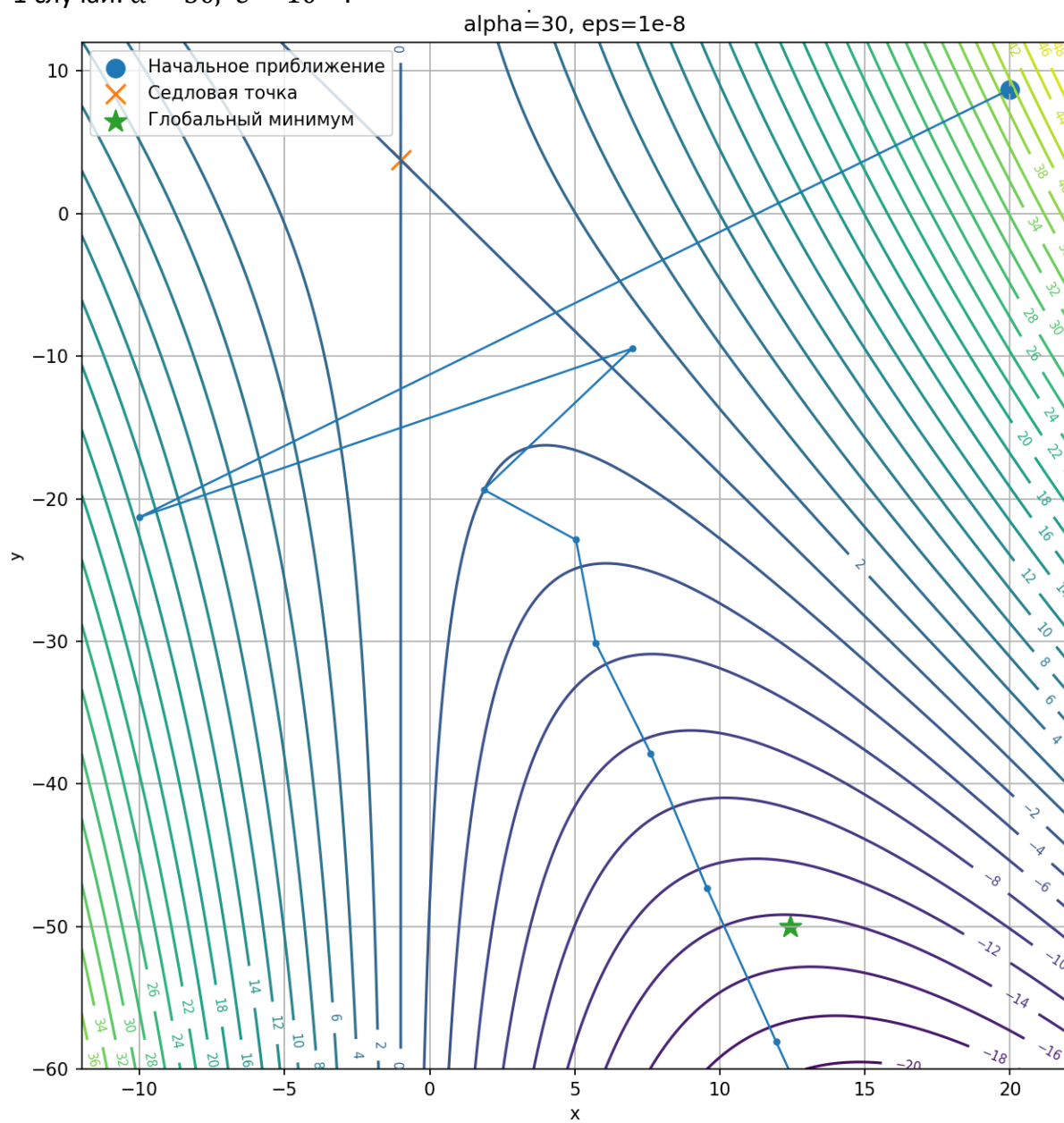
2.3. Подбор области отображения и гиперпараметров

Для наглядного сравнения траекторий была выбрана одна область отображения: $x \in [-12, 22], y \in [-60, 12]$.

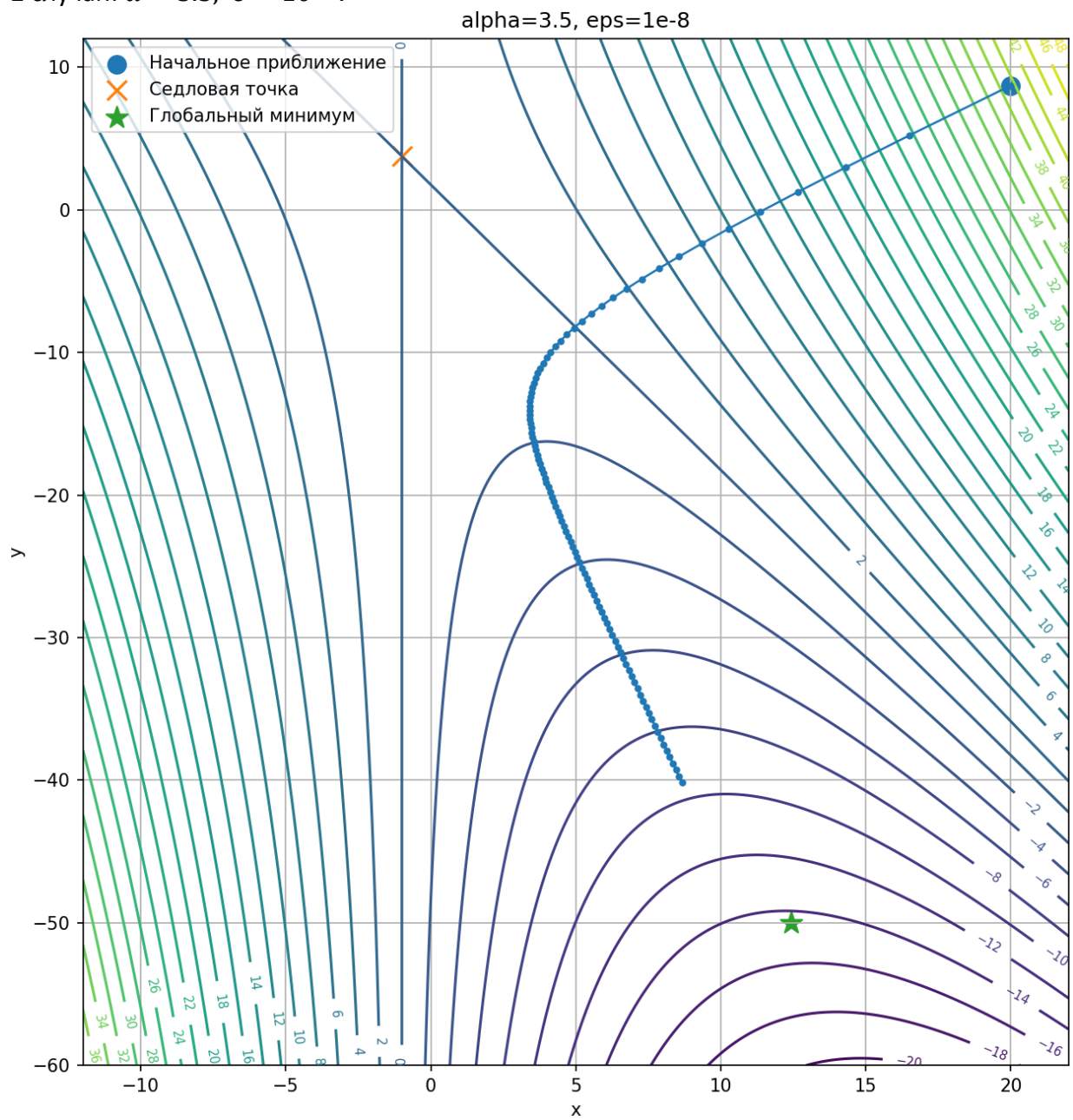
В этой области хорошо видны начальная точка, седловая точка, глобальный минимум и характер движения алгоритма.

Были подобраны два набора гиперпараметров Adagrad.

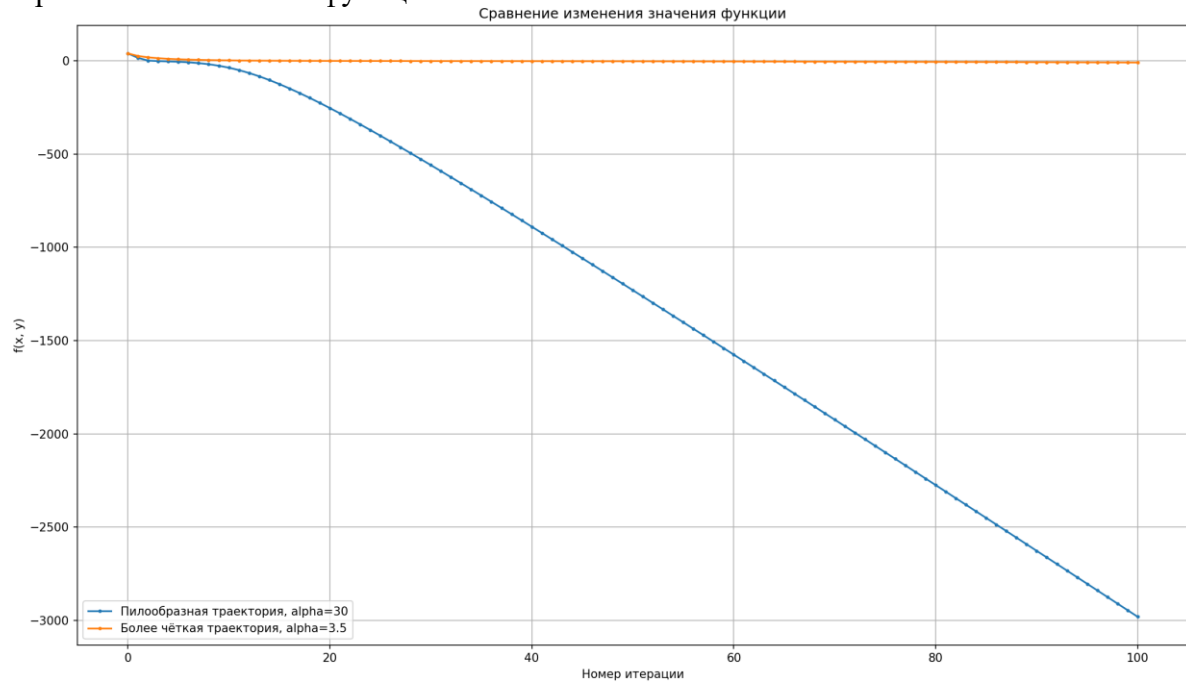
1 случай: $\alpha = 30$, $\varepsilon = 10^{-8}$.



2 случай: $\alpha = 3.5$, $\varepsilon = 10^{-8}$.



Сравнение изменения функции:



2.4. Готовая реализация Adagrad из PyTorch

Для проверки результатов была использована готовая реализация метода Adagrad из библиотеки PyTorch:

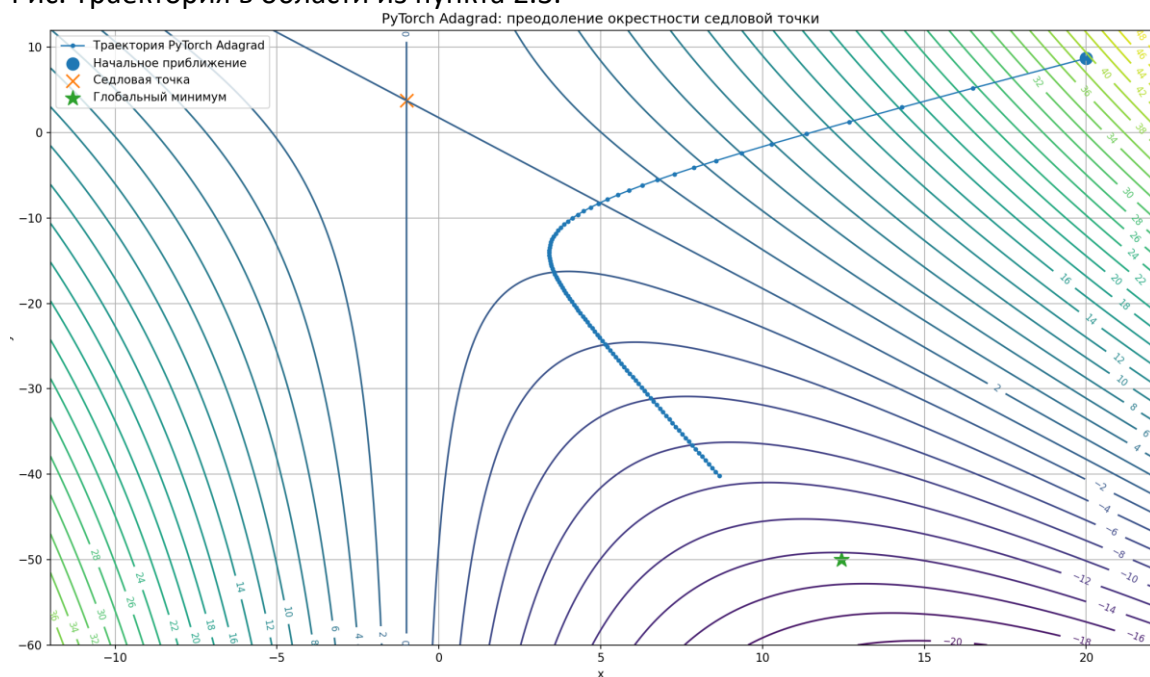
Оптимизация выполнялась для той же функции, с той же начальной точкой и тем же числом итераций: $(x_0, y_0) \approx (20, 8.7074)$, $N = 100$.

Параметры оптимизатора:

$$lr = 3.5, \quad \text{eps} = 10^{-8}.$$

Готовая реализация PyTorch также не остановилась в седловой точке и продолжила движение в сторону глобального минимума.

Рис. Траектория в области из пункта 2.3.



Библиотечная реализация `torch.optim.Adagrad` успешно решает задачу преодоления окрестности седловой точки. Её поведение согласуется с собственной реализацией `Adagrad`.

Вывод по заданию

В ходе выполнения задания было показано различие между обычным градиентным спуском и методом `Adagrad`. Обычный градиентный спуск при выбранных параметрах сошёл к седловой точке: $(-1, 3.75)$. Это демонстрирует проблему застревания в окрестности седловых точек.

Собственная и готовая реализация смогли преодолеть окрестность седловой точки и продолжили движение в сторону глобального минимума. Это объясняется адаптивной по координатной скоростью обучения, которая учитывает накопленную историю квадратов градиентов. Также было показано, что выбор гиперпараметра α существенно влияет на форму траектории.

Задание 3. Обучение нейронной сети и сравнение оптимизаторов

В качестве датасета был выбран `Human Activity Recognition Using Smartphones 2012` г. Датасет содержит данные, полученные с акселерометра и гироскопа смартфона. По этим данным необходимо классифицировать активность человека.

Характеристики датасета:

Количество объектов 10 299

Количество признаков 561

Классы активности: 0:WALKING, 1:WALKING_UPSTAIRS, 2:WALKING_DOWNSTAIRS, 3:SITTING, 4:STANDING, 5:LAYING.

Предобработка данных

В датасете уже присутствовало официальное разделение на обучающую и тестовую выборки.

Размеры выборок:

обучающая выборка: `X_train: (7352, 561) y_train: (7352,)`

тестовая выборка: `X_test : (2947, 561) y_test : (2947,)`

Этапы предобработки:

1. Были загружены обучающая и тестовая выборки.
2. Признаки были нормализованы с помощью `StandardScaler`.
3. `StandardScaler` обучался только на обучающей выборке, после чего применялся к `train` и `test`.
4. Метки классов были перекодированы из диапазона 1..6 в диапазон 0..5.
5. Данные были преобразованы в `torch.Tensor` и переданы в `DataLoader`.

Архитектура нейронной сети

Была создана полносвязная нейронная сеть `ActivityMLP`.

Архитектура: $561 \rightarrow 128 \rightarrow 64 \rightarrow 6$

Входной слой получает 561 признак. Выходной слой содержит 6 нейронов, так как решается задача классификации на 6 классов.

Метрики качества:

- Test loss — значение функции потерь на тестовой выборке. Показывает, насколько сильно предсказания модели отличаются от правильных ответов. Чем меньше Test loss, тем лучше.
- Accuracy — доля правильно классифицированных объектов.
- Macro F1-score — среднее значение F1-score по всем классам. Учитывает и точность, и полноту для каждого класса, а затем усредняет результат.

Параметры обучения

Для обучения использовались одинаковые параметры для библиотечного и собственного оптимизатора.

Параметр	Значение
Количество эпох	50
Learning rate	0.1
eps	10^{-10}
weight decay	10^{-4}

Обучение с библиотечным Adagrad

Сначала модель была обучена с готовым оптимизатором PyTorch.

Результаты на тестовой выборке:

Метрика	Значение
Test loss	0.3633
Accuracy	0.8972
Macro F1	0.8951

Собственная реализация CustomAdagrad

Оптимизатор наследуется от torch.optim.Optimizer, поэтому используется так же, как стандартные оптимизаторы PyTorch:

Формулы метода:

$$G_{k+1} = G_k + g_k^2$$

$$w_{k+1} = w_k - \frac{lr}{\sqrt{G_{k+1}} + \epsilon} g_k$$

Использованные параметры:

$$lr = 0.1, \quad \text{eps} = 1e - 10, \quad \text{weight_decay} = 1e - 4$$

Обучение с CustomAdagrad

Та же модель была обучена с собственной реализацией Adagrad.

Результаты на тестовой выборке:

Метрика	Значение
---------	----------

Test loss	0.3633
Accuracy	0.8972
Macro F1	0.8951

Сравнение оптимизаторов

Сравним результаты библиотечной и собственной реализации Adagrad.

Оптимизатор	Test loss	Accuracy	Macro F1
torch.optim.Adagrad	0.3633	0.8972	0.8951
CustomAdagrad	0.3633	0.8972	0.8951

Результаты совпали. Это объясняется тем, что в CustomAdagrad была реализована та же схема накопления квадратов градиентов, использовались одинаковые гиперпараметры, одинаковая архитектура модели и одинаковое начальное состояние.

Полученное значение Accuracy = 0.8972 означает, что модель правильно классифицирует около 90% объектов тестовой выборки. Значение MacroF1 = 0.8951 близко к accuracy, значит качество классификации достаточно равномерное по классам. Наиболее вероятные ошибки возникают между похожими активностями, например SITTING и STANDING, так как эти состояния могут иметь близкие показания датчиков смартфона.

Вывод по заданию

В задании была решена задача классификации активности человека по 561 признаку.

Модель была обучена двумя способами:

с библиотечным оптимизатором torch.optim.Adagrad;

с собственной реализацией CustomAdagrad.

Оба оптимизатора показали одинаковые результаты: Accuracy = 0.8972, MacroF1 = 0.8951. Следовательно, собственная реализация Adagrad была корректно интегрирована в процесс обучения нейронной сети.

Итоговый вывод по работе

В ходе лабораторной работы была исследована задача оптимизации функции двух переменных с седловой точкой. Было показано, что обычный градиентный спуск может сходиться не к глобальному минимуму, а к седловой точке, если начальное приближение и параметры подобраны соответствующим образом.

Для преодоления этой проблемы был реализован метод Adagrad, использующий адаптивную по координатам скорость обучения. Собственная реализация Adagrad и готовая реализация torch.optim.Adagrad показали успешное прохождение окрестности седловой точки и движение в сторону глобального минимума.

Также была решена задача классификации активности человека на датасете Human Activity Recognition Using Smartphones. Для неё была построена нейронная сеть и выполнено обучение с библиотечным и собственным оптимизатором Adagrad.

Результаты двух оптимизаторов совпали, что подтверждает корректность собственной реализации CustomAdagrad. Работа показала, что адаптивные методы оптимизации лучше справляются со сложным ландшафтом функции потерь, чем обычный градиентный спуск.